

Emotion Detection & Learning Support Engine

Project Description

Emotion Detection & Personalized Learning Support Platform

The AI-Driven Emotion Detection & Personalized Learning Support Platform is an end-to-end intelligent learning assistant that analyzes a learner's free-text description of academic challenges and provides emotion-aware guidance. The system combines BiLSTM and BERT deep learning models with rule-based keyword enhancement to detect emotions such as Bored, Confident, Confused, Curious, and Frustrated. Based on the detected emotional state, Google Gemini generates personalized learning guidance, study strategies, and motivational responses.

The platform provides real-time emotion prediction, mixed-emotion analysis, model comparison, confidence scores, analytics dashboards, and interaction history. Built using Python and Streamlit, it enables students, educators, and academic support teams to understand emotional learning barriers and receive consistent, empathetic, AI-powered assistance. The system improves learning engagement, reduces response time, and enhances personalized educational support.

Scenario 1 – Educators & Academic Support Teams

Educators, teaching assistants, and academic support staff often spend considerable time interpreting student concerns before providing assistance. The platform automates this process by analyzing students' text descriptions, detecting their emotional state, and generating personalized learning guidance. For example, if a student expresses frustration about mathematics, the system classifies the emotion, compares predictions from BiLSTM and BERT, identifies mixed emotions when applicable, and generates supportive recommendations using Gemini AI. This enables educators to respond faster while maintaining consistent and empathetic support.

Scenario 2 – Students & Self-Learning

Students preparing independently frequently experience confusion, stress, or low confidence during their studies. Instead of searching through multiple resources, they can simply describe their learning difficulty in natural language. The platform analyzes the text, predicts the emotional state, and provides personalized guidance, motivation, and recommended learning strategies. It also stores previous interactions and presents analytics, allowing students to monitor emotional trends and learning progress over time.

Technologies Used

- Python
- Streamlit
- TensorFlow / Keras
- BiLSTM

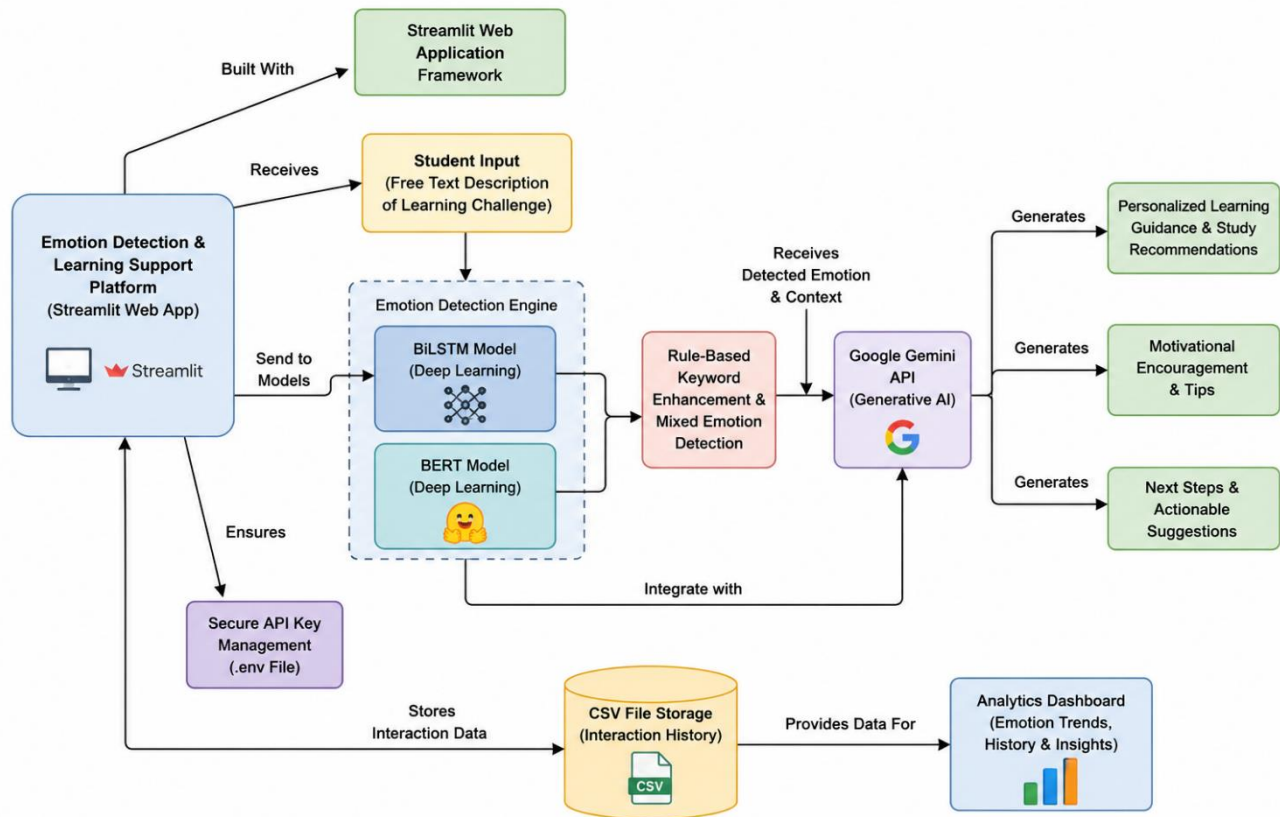
- BERT
- Google Gemini API
- Pandas
- NumPy
- Matplotlib
- CSV Data Storage

Key Features

- AI-based text emotion detection
- Dual-model prediction (BiLSTM & BERT)
- Mixed emotion detection
- Confidence score visualization
- Gemini-powered personalized learning guidance
- Subject-wise learning support
- Analytics dashboard
- Interaction history
- Real-time emotion insights
- Streamlit web application

's gone!" to maximize conversion.

Technical Architecture:



Description

The Emotion Detection & Personalized Learning Support Platform utilizes a modular AI-driven architecture to deliver real-time emotion analysis and personalized learning assistance. The frontend is developed using Streamlit, providing an interactive and user-friendly interface for students to describe their learning challenges. The backend integrates BiLSTM and BERT deep learning models for text-based emotion detection, enhanced by a rule-based keyword analysis module to improve prediction accuracy and identify mixed emotions. The detected emotional context is then passed to the Google Gemini API, which generates personalized study guidance, motivational support, and actionable learning recommendations. User interactions are securely stored in CSV files for history tracking and analytics visualization, while the application is deployed on Streamlit Community Cloud for seamless web-based access.

Pre-requisites

- **Python Programming Knowledge:** Python Documentation
- **Streamlit Framework Knowledge:** Streamlit Documentation
- **TensorFlow / Keras Knowledge:** TensorFlow Documentation
- **BERT & BiLSTM Model Familiarity:** Hugging Face & TensorFlow Documentation

- **Google Gemini API Familiarity:** Google AI Studio Documentation
- **Pandas & NumPy Knowledge:** Pandas and NumPy Documentation
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Visual Studio Code & Python Environment
- **Streamlit Community Cloud Deployment:** Streamlit Deployment Documentation

Project Workflow

Milestone 1: Project Planning & Model Selection

- Activity 1.1: Define the project objectives, user requirements, and supported emotion categories (Bored, Confident, Confused, Curious, Frustrated).
- Activity 1.2: Research and select suitable deep learning models (BiLSTM and BERT) for text-based emotion detection.
- Activity 1.3: Configure the Google Gemini API securely for generating personalized learning guidance.
- Activity 1.4: Set up the development environment by installing Python, Streamlit, TensorFlow/Keras, Pandas, NumPy, and other required libraries.

Milestone 2: Emotion Detection Module Development

- Activity 2.1: Develop the text preprocessing pipeline for cleaning and preparing student input.
- Activity 2.2: Implement the BiLSTM and BERT models for emotion classification.
- Activity 2.3: Integrate rule-based keyword enhancement and mixed-emotion detection to improve prediction accuracy.

Milestone 3: AI Learning Support Integration

- Activity 3.1: Integrate the Google Gemini API to generate personalized learning guidance based on detected emotions.
- Activity 3.2: Develop the recommendation logic to provide motivational support, study tips, and actionable learning strategies.

Milestone 4: Streamlit Application Development

- Activity 4.1: Design and develop the interactive user interface using Streamlit.
- Activity 4.2: Implement subject selection, text input, emotion visualization, model comparison, analytics dashboard, and interaction history.
- Activity 4.3: Display BiLSTM and BERT predictions, confidence scores, mixed emotions, and AI-generated guidance in real time.

Milestone 5: Testing & Deployment

- Activity 5.1: Perform functional testing, validate emotion predictions, and verify AI-generated recommendations.
- Activity 5.2: Deploy the application on Streamlit Community Cloud and verify end-to-end functionality.
- Activity 5.3: Store user interactions in CSV files for analytics and history tracking.

Milestone 6: Project Completion & Documentation

- Activity 6.1: Prepare the final project documentation, architecture diagrams, and user guide.
- Activity 6.2: Record the project demonstration video and upload the source code to GitHub.
- Activity 6.3: Review the application, finalize documentation, and submit the completed project.

Milestone 1: Project Planning & Model Selection

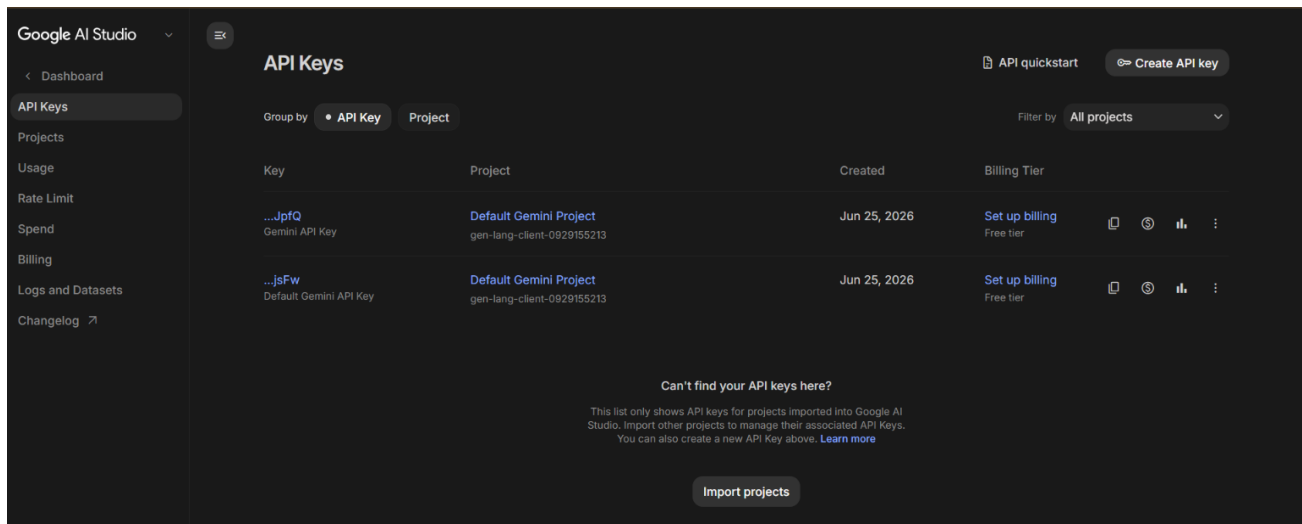
In this milestone, the primary focus is on planning the overall system architecture, selecting suitable Artificial Intelligence models for emotion detection, and preparing the development environment. The project aims to analyze students' textual descriptions of their learning challenges and provide personalized learning support based on the detected emotional state. To achieve this objective, appropriate Natural Language Processing (NLP) models, cloud-based AI services, and deployment technologies were researched and selected before development began.

Activity 1.1: Configure Google Gemini API

Before the application can generate personalized learning guidance, a valid Google Gemini API key must be obtained and configured securely within the project.

Steps

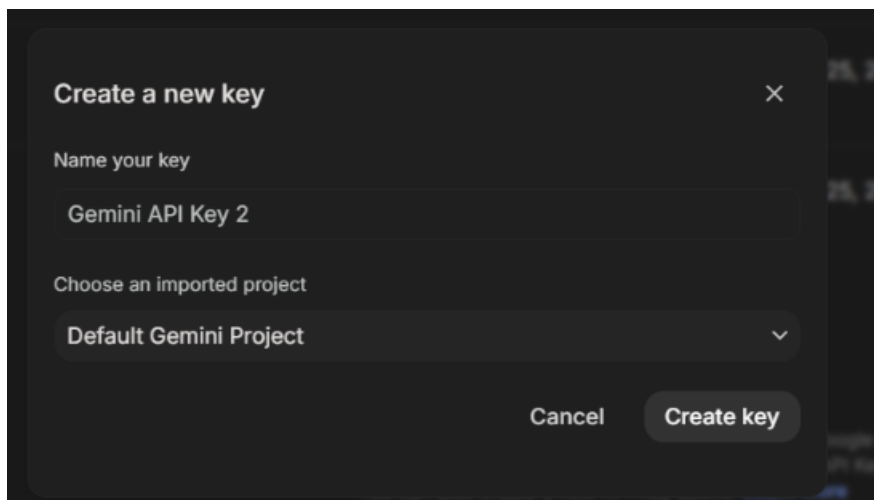
- Create a Google AI Studio account.



Navigate to **API Keys**.

 **Image:** Screenshot showing **API Key section**

- Generate a new API Key.



- Create a **.env** file inside the project directory.

```
.env
# Gemini API configuration
GEMINI_API_KEY=AQ.Ab8RN6JgVNf-TKwfby00fjkIDhgAu1PyhQzRzpZC8vXEzVJp
# Session and storage settings
CSV_LOG_PATH=data/logs/session_history.csv
# Model paths
BILSTM_MODEL_PATH=models/bilstm/bilstm_model.h5
BERT_MODEL_PATH=models/bert/bert_model
TOKENIZER_PATH=models/tokenizer
# Analytics settings
MAX_HISTORY_RECORDS=500
```

Load the API securely inside **app.py** using **python-dotenv**

```
config = load_config()
except Exception as exc:
    st.error(f"Configuration load failed: {exc}")
    return

SessionManager.initialize_state()
selected_field = render_sidebar(SessionManager.get_field())
SessionManager.set_field(selected_field)

try:
    predictor = get_predictor(config)
    gemini_generator = get_gemini_generator(config)
    csv_logger = get_csv_logger(config)
except Exception as exc:
    logger.exception("Failed to initialize application resources.")
    st.error(f"Application initialization failed: {exc}")
    return
```

Activity 1.2: Research and Select Emotion Detection Models

The success of the platform depends on accurately identifying students' emotional states from textual descriptions. Various Natural Language Processing approaches were studied before selecting the final models.

The following factors were considered during model selection:

- Emotion classification accuracy
- Context understanding
- Computational efficiency
- Compatibility with educational text
- Ease of deployment

After evaluation, the following models were selected:

BiLSTM

BiLSTM captures sequential relationships within text and performs efficient emotion classification by understanding contextual dependencies from both forward and backward directions.

BERT

BERT provides contextual language understanding using transformer-based bidirectional attention mechanisms, enabling more accurate prediction of complex emotional expressions.

Both model predictions are compared, and rule-based keyword enhancement is applied to identify mixed emotions and improve the overall reliability of emotion detection.

Student Text

↓

BiLSTM

↓

Emotion Prediction

AND

Student Text

↓

BERT

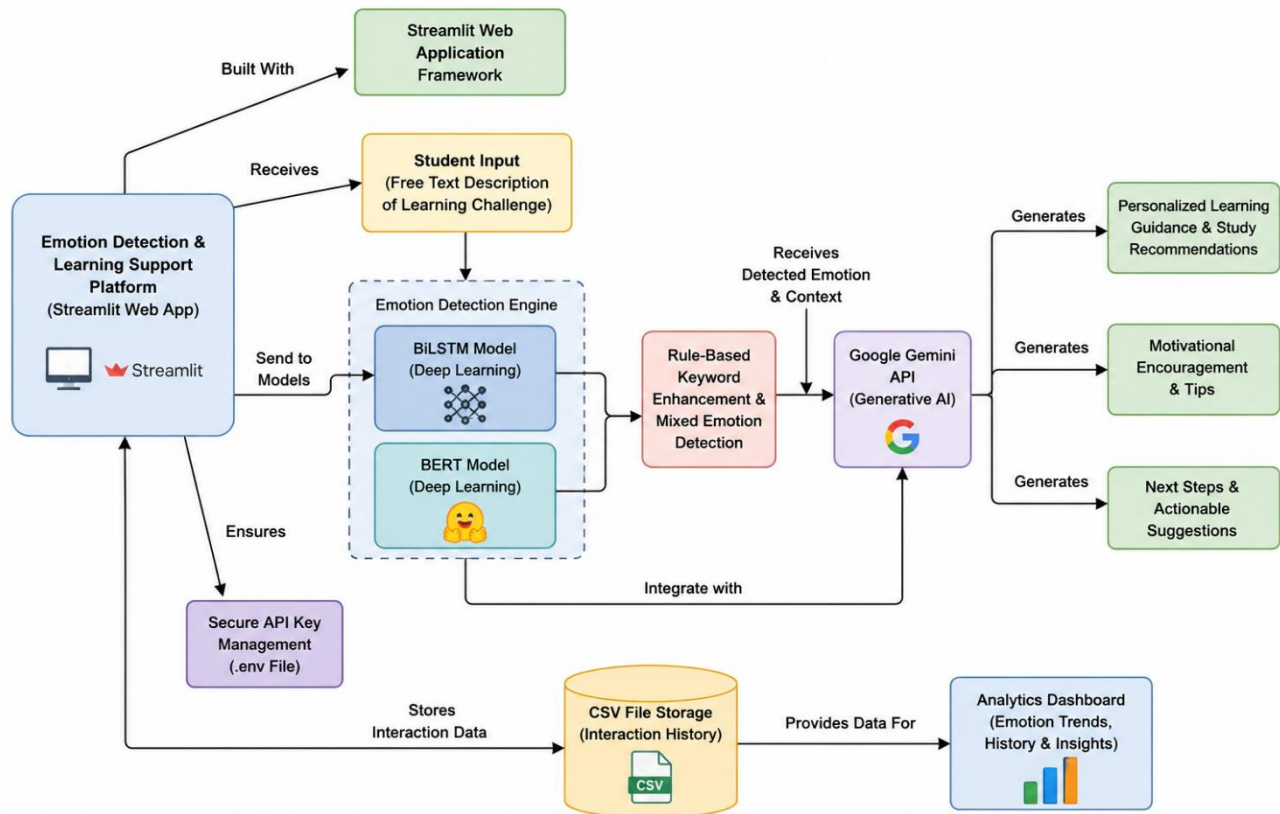
↓

Emotion Prediction

Activity 1.3: Define the Architecture of the Application

The Emotion Detection & Personalized Learning Support Platform follows a modular AI-driven architecture consisting of multiple interconnected components.

The Streamlit interface allows students to describe their learning challenges in natural language. The input text is processed simultaneously by the BiLSTM and BERT emotion detection models. Rule-based keyword enhancement combines the predictions and identifies mixed emotions. The final emotional context is then passed to the Google Gemini API, which generates personalized learning guidance, motivational support, and study recommendations. User interactions are stored in CSV files for history tracking and analytics visualization through the dashboard.



Activity 1.4: Set Up the Development Environment

The development environment was configured using Python and Streamlit. Required libraries, including TensorFlow, Keras, Pandas, NumPy, Plotly, and Google Gemini integration modules, were installed before development. The application was developed using Visual Studio Code and deployed through Streamlit Community Cloud.

The project was executed locally using the following command:

```
python -m streamlit run app.py
```

After execution, the Streamlit server launches the application locally, enabling users to interact with the AI-powered learning assistant through a web browser.

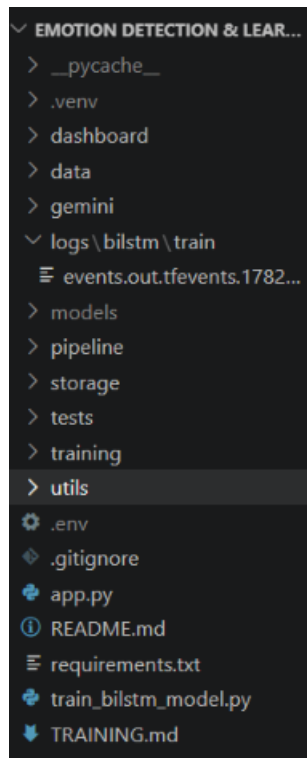
```
PS C:\Users\rithv\OneDrive\Desktop\Emotion Detection & Learning Support Engine> python -m streamlit run app.py
2026-07-04 12:19:25.168 Uvicorn server started on 0.0.0.0:8501
```

```
You can now view your Streamlit app in your browser.
```

```
Local URL: http://localhost:8501
```

```
Network URL: http://192.168.29.125:8501
```

Project directory structure in Visual Studio Code



Milestone 2: Core Functionalities Development

This milestone focuses on developing the core functionalities of the Emotion Detection & Personalized Learning Support Platform. The primary objective is to process students' textual descriptions of learning challenges, accurately detect their emotional state using deep learning models, and generate personalized learning guidance. The platform integrates multiple AI components to ensure reliable emotion prediction, mixed emotion detection, and supportive recommendations.

Activity 2.1: Develop the Core Emotion Detection Features

The core functionality of the application was developed to analyze students' textual input and identify their emotional state using Artificial Intelligence. The following features were implemented:

Text Emotion Detection

- Accepts free-text descriptions of students' learning challenges.
- Preprocesses and prepares the text for emotion prediction.
- Detects emotions such as **Bored, Confident, Confused, Curious, and Frustrated**.

BiLSTM Emotion Prediction

- Uses the BiLSTM deep learning model to classify emotions from the processed text.
- Generates prediction probabilities and confidence scores for each emotion category.

BERT Emotion Prediction

- Uses the BERT transformer model to perform contextual emotion analysis.
- Provides highly accurate emotion predictions by understanding semantic relationships within the text.

Mixed Emotion Detection

- Compares predictions from both models.
- Applies rule-based keyword enhancement to identify multiple emotional states when appropriate.
- Produces a final emotion prediction with an associated confidence score.

Activity 2.2: Integrate AI-Based Personalized Learning Support

The application integrates the Google Gemini API to generate supportive and personalized learning guidance based on the detected emotional state.

Emotion-Aware Guidance Generation

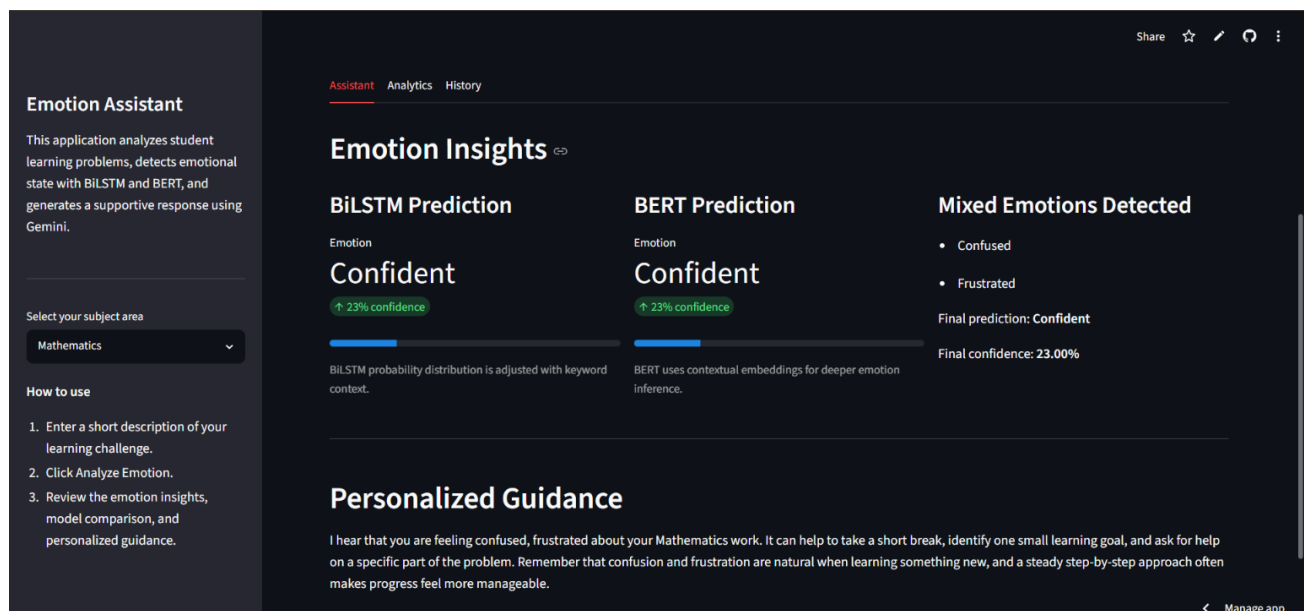
- Sends the final predicted emotion and student context to the Gemini AI model.
- Generates empathetic responses that address the student's emotional condition.

Personalized Learning Recommendations

- Suggests effective study strategies based on the detected emotion.
- Encourages students with motivational messages and actionable next steps.
- Provides subject-specific learning support tailored to the student's academic challenge.

Interaction Logging

- Stores each prediction and generated response in CSV files.
- Maintains interaction history for future analysis and visualization through the analytics dashboard.



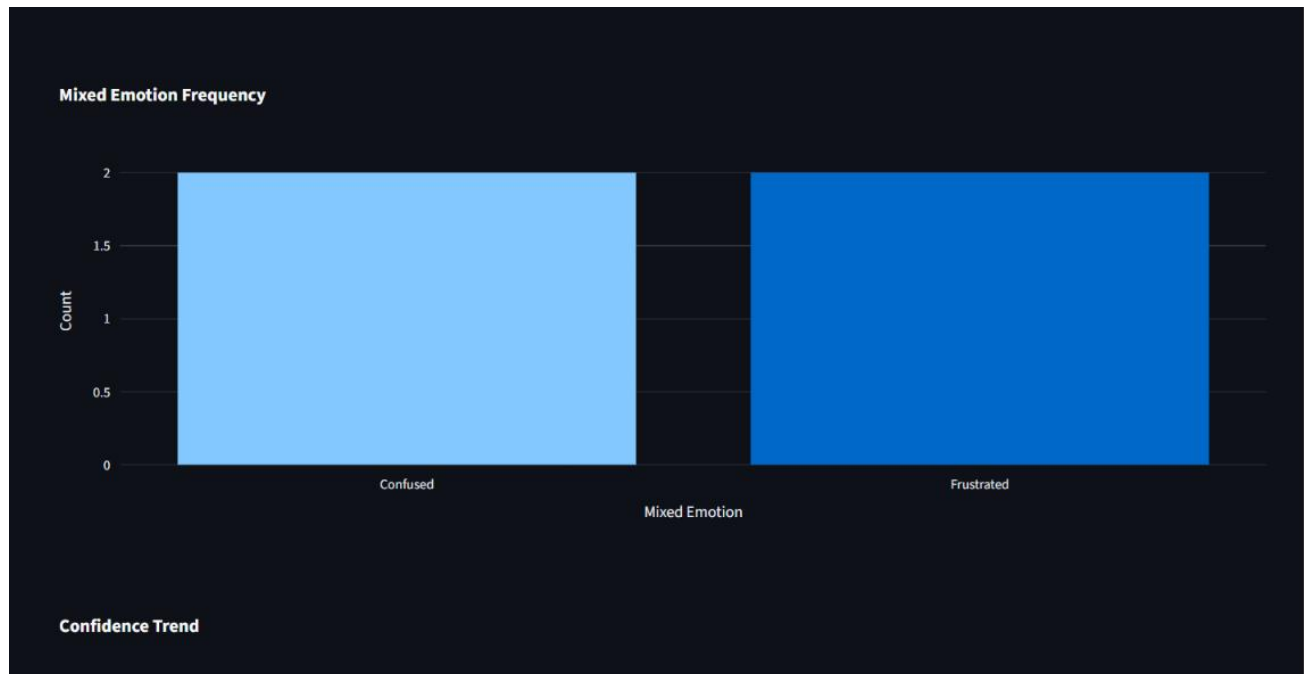
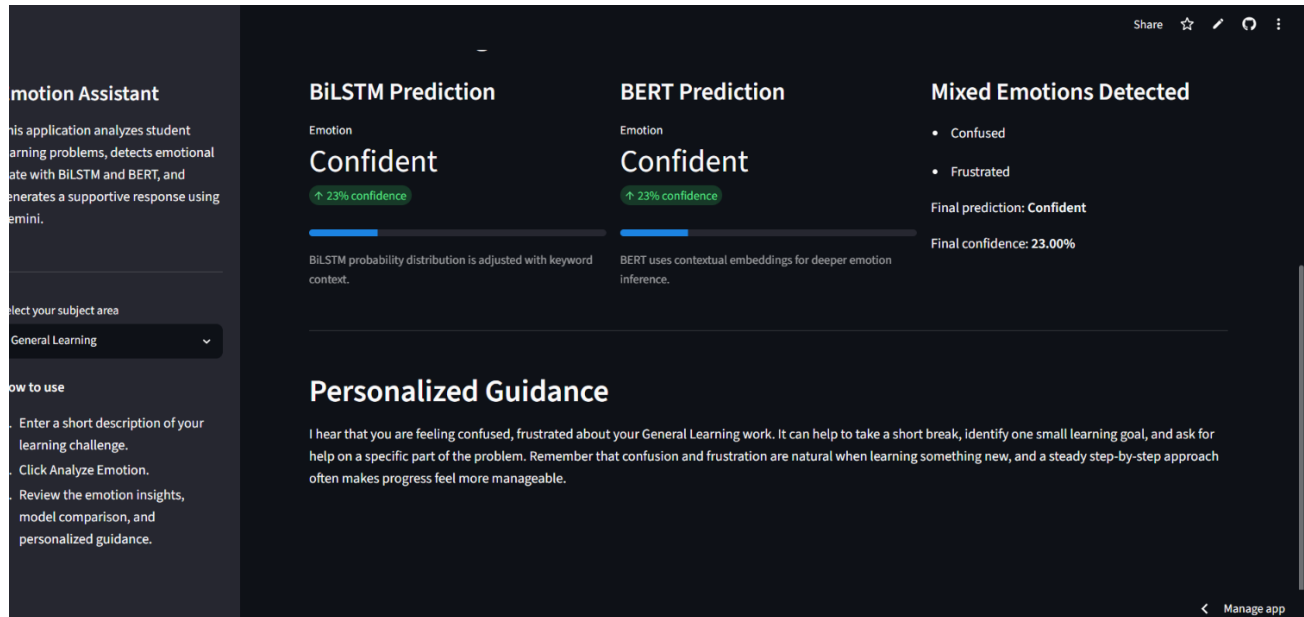
The screenshot displays the 'Emotion Assistant' application interface. On the left, a sidebar contains the title 'Emotion Assistant', a description of the application's function (analyzing student learning problems, detecting emotional state with BiLSTM and BERT, and generating supportive responses using Gemini), a subject selection dropdown menu currently set to 'Mathematics', and a 'How to use' section with three numbered steps: 1. Enter a short description of your learning challenge, 2. Click Analyze Emotion, and 3. Review the emotion insights, model comparison, and personalized guidance.

The main content area is titled 'Emotion Insights' and features three columns of data:

- BiLSTM Prediction:** Shows an emotion of 'Confident' with a '23% confidence' score, represented by a blue progress bar.
- BERT Prediction:** Also shows an emotion of 'Confident' with a '23% confidence' score, represented by a blue progress bar.
- Mixed Emotions Detected:** Lists 'Confused' and 'Frustrated' as detected emotions. It provides a 'Final prediction: Confident' and a 'Final confidence: 23.00%'.

Below these predictions, there is a section for 'Personalized Guidance' which contains a supportive message: 'I hear that you are feeling confused, frustrated about your Mathematics work. It can help to take a short break, identify one small learning goal, and ask for help on a specific part of the problem. Remember that confusion and frustration are natural when learning something new, and a steady step-by-step approach often makes progress feel more manageable.'

At the bottom right of the interface, there is a 'Manage app' link.



Milestone 3: Main Application Development

Milestone 3 focuses on developing the core application logic in app.py, which serves as the central component of the Emotion Detection & Personalized Learning Support Platform. In this milestone, the application initializes the required AI models, processes user input, performs emotion prediction using BiLSTM and BERT, generates personalized learning guidance through Google Gemini, stores interaction history, and displays analytics through an interactive Streamlit interface.

Activity 3.1: Develop the Main Application Logic (app.py)

The app.py file acts as the entry point of the application and coordinates all major functionalities. It initializes the required components, handles user interactions, processes emotion predictions, and displays personalized learning guidance.

The following tasks were implemented:

Configure the Application

- Load project configuration settings.
- Initialize the Emotion Predictor, Gemini Response Generator, and CSV Logger.
- Configure the Streamlit application page and sidebar.

Process Student Input

- Accept textual descriptions of learning challenges.
- Allow students to select their academic subject.
- Validate the input before emotion analysis.

Emotion Prediction

- Process the student's text using the BiLSTM and BERT emotion detection models.
- Compare both model predictions.
- Detect mixed emotions and calculate confidence scores.
- Generate the final emotion prediction.

Generate Personalized Guidance

- Send the detected emotional context to Google Gemini.
- Generate supportive learning guidance and study recommendations.
- Display the personalized response within the application.

Store Prediction History

- Save prediction results and generated responses to CSV files.
- Maintain session history for future reference.
- Support analytics and visualization of previous interactions.

Analytics and History

- Display emotion trends and confidence scores.
- Compare BiLSTM and BERT model predictions.
- Visualize interaction history using charts and dashboards.

```
try:
    config = load_config()
except Exception as exc:
    st.error(f"Configuration load failed: {exc}")
    return

SessionManager.initialize_state()
selected_field = render_sidebar(SessionManager.get_field())
SessionManager.set_field(selected_field)

try:
    predictor = get_predictor(config)
    gemini_generator = get_gemini_generator(config)
    csv_logger = get_csv_logger(config)
except Exception as exc:
    logger.exception("Failed to initialize application resources.")
    st.error(f"Application initialization failed: {exc}")
    return
```

Figure 3.1: Initialization of the application by loading the project configuration, setting up the session state, and initializing the Emotion Predictor, Gemini Response Generator, and CSV Logger.

```
st.header("Describe your learning challenge")
student_input = st.text_area(
    "Student problem description",
    value=st.session_state.get("student_input", ""),
    height=220,
    placeholder="I feel confused about the latest assignment because...",
    key="student_input",
)
```

Figure 3.2: Streamlit interface for accepting student input and initiating emotion analysis.

```
def run_analysis(
    user_input: str,
    selected_field: str,
    predictor: EmotionPredictor,
    gemini_generator: GeminiResponseGenerator,
    csv_logger: CsvLogger,
) -> None:
    if not user_input.strip():
        SessionManager.set_error("Please enter a learning problem to analyze.")
        return

    try:
        with st.spinner("Analyzing emotion and generating guidance..."):
            prediction = predictor.predict(user_input, selected_field)
            prediction.gemini_response = gemini_generator.generate_response(prediction)
            csv_logger.validate_record(prediction.to_dict())
            csv_logger.append_record(prediction.to_dict())
            SessionManager.add_prediction(prediction)
            SessionManager.set_error("")
    except Exception as exc:
        logger.exception("Prediction or logging failed.")
        SessionManager.set_error("Unable to complete analysis. Check model files and .env settings.")
```

Figure 3.3: Execution of the emotion analysis workflow, including emotion prediction, AI-generated personalized guidance, validation, logging, and session management.

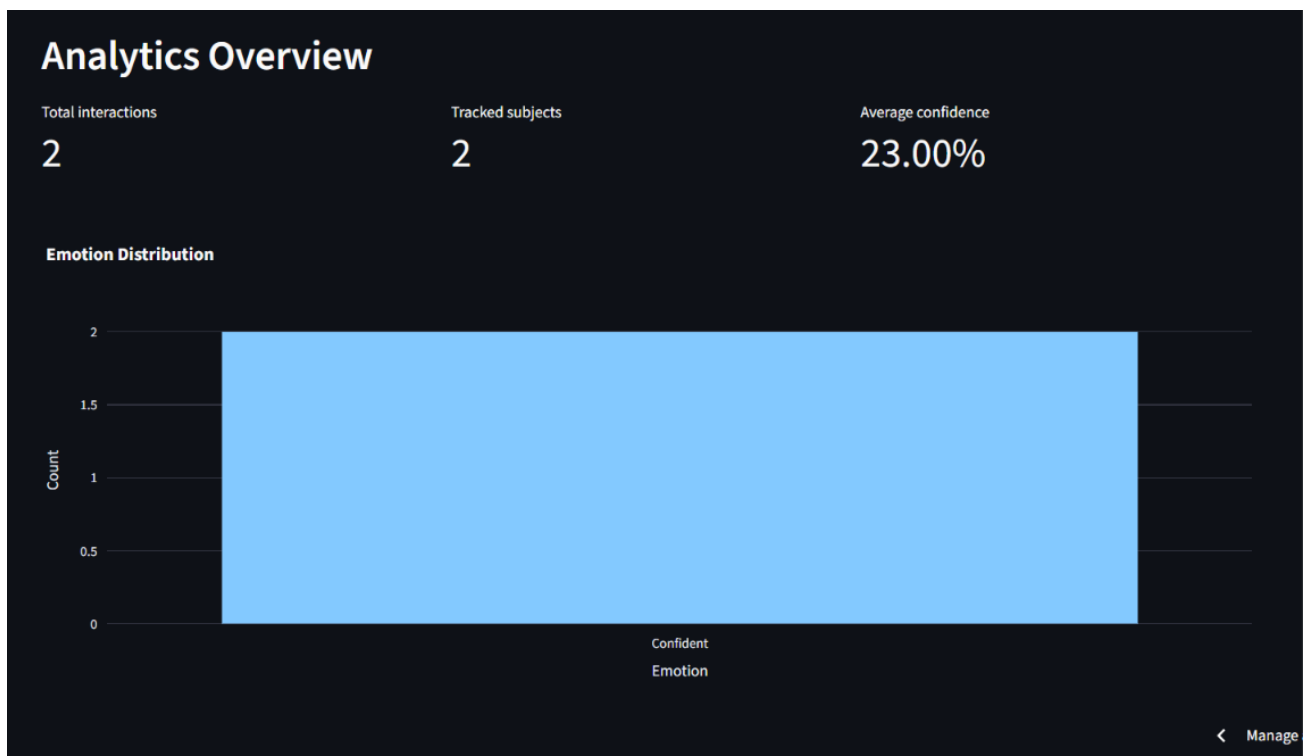


Figure 3.4: Analytics dashboard displaying emotion trends

Milestone 4: User Interface Development

Milestone 4 focuses on developing an intuitive and interactive user interface for the **Emotion Detection & Learning Support Engine** using the **Streamlit** framework. The interface allows students to describe their learning challenges, receive AI-powered emotion analysis, compare predictions from BiLSTM and BERT models, and obtain personalized learning guidance generated through Google Gemini. This milestone also includes designing a responsive layout, implementing dynamic content updates, and providing analytics and history tracking for an enhanced user experience.

Activity 4.1: Designing and Developing the User Interface

Set Up the Main Streamlit Interface

- Develop the main Streamlit application (**app.py**) as the central interface for the platform.
- Design a clean and organized layout consisting of a sidebar, input section, emotion insights panel, analytics dashboard, and history page.
- Configure the Streamlit page using `st.set_page_config()` to provide an optimized layout and responsive user experience.

Design an Interactive Sidebar

- Create a sidebar containing the project description and application overview.
- Provide a subject selection dropdown allowing students to choose their academic field.
- Display clear usage instructions explaining how to interact with the application.

Create Student Input Components

- Implement a large text input area where students can describe their learning challenges in natural language.
- Add an **Analyze Emotion** button to begin emotion prediction.
- Provide a **Clear Session** button to reset the current analysis and clear previous inputs.

Display Emotion Prediction Results

- Display emotion predictions generated by both the **BiLSTM** and **BERT** models.
- Present confidence scores using progress indicators.
- Show mixed emotion detection results along with the final predicted emotion.
- Display personalized learning guidance generated using **Google Gemini**.

Activity 4.2: Creating Dynamic Content Rendering with Streamlit

Process Student Input Dynamically

- Accept student input through the Streamlit interface and validate it before processing.
- Display a loading spinner while emotion prediction and AI guidance generation are

being performed.

- Process the input using BiLSTM and BERT models and generate personalized recommendations in real time.

Render Results Dynamically

- Display emotion predictions immediately after successful analysis.
- Present BiLSTM prediction, BERT prediction, mixed emotions, final confidence score, and AI-generated learning guidance in a structured layout.
- Update the interface dynamically without requiring manual page refreshes.

Provide Analytics and History

- Display emotion distribution, confidence trends, subject-wise analysis, and model comparison through interactive charts.
- Maintain a history of previous predictions using CSV storage.
- Allow users to review previous interactions and download prediction history for future reference.

Maintain User Session

- Preserve user input and prediction results during the active session.
- Allow users to reset the application state using the **Clear Session** option.
- Ensure smooth navigation between the **Assistant**, **Analytics**, and **History** sections while maintaining application responsiveness.

Milestone 5: Deployment

Milestone 5 focuses on deploying the Emotion Detection & Learning Support Engine locally for testing and on Streamlit Community Cloud for public access. This milestone includes configuring the project environment, installing required dependencies, verifying application functionality, and deploying the application for online accessibility.

Activity 5.1: Preparing the Application for Local Deployment

Replace Flask instructions with:

Include

- Install Python 3.10+
- Open the project folder
- Install dependencies

`pip install -r requirements.txt`

- Configure Gemini API Key

Create

`.env`

Add

GEMINI_API_KEY=your_api_key

Run

python -m streamlit run app.py

```
PS C:\Users\rithv\OneDrive\Desktop\Emotion Detection & Learning Support Engine> python -m streamlit run app.py
2026-07-04 12:19:25.168 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.29.125:8501
```

Activity 5.2: Local Testing & Verification

After successfully running the application using Streamlit, the local server launches the Emotion Detection & Learning Support Engine in the default web browser. The application was tested by entering various student learning challenges, selecting different academic subjects, and verifying the correctness of emotion prediction, mixed emotion detection, personalized learning guidance, analytics, and history tracking.

Test:

- Mathematics
- Computer Science
- AI
- Physics

Test emotions

- Confused
- Frustrated
- Curious
- Confident

Verify

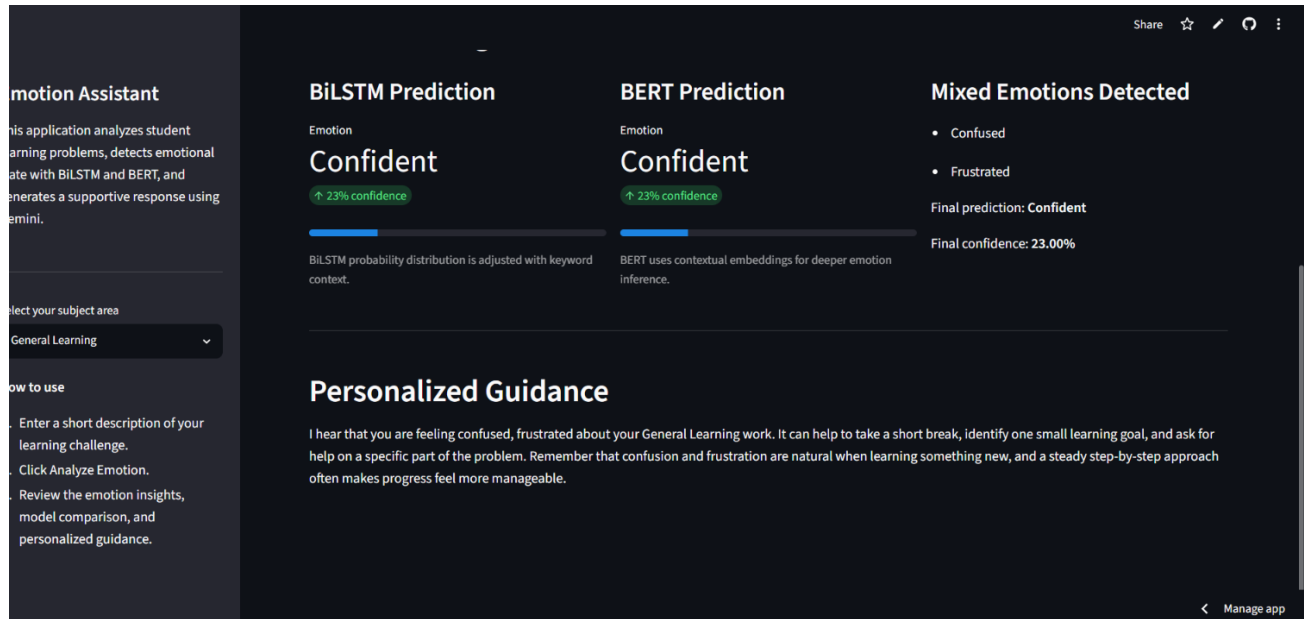
✓ BiLSTM

✓ BERT

✓ Gemini

✓ Analytics

✓ History



Activity 5.3: Deployment on Streamlit Community Cloud

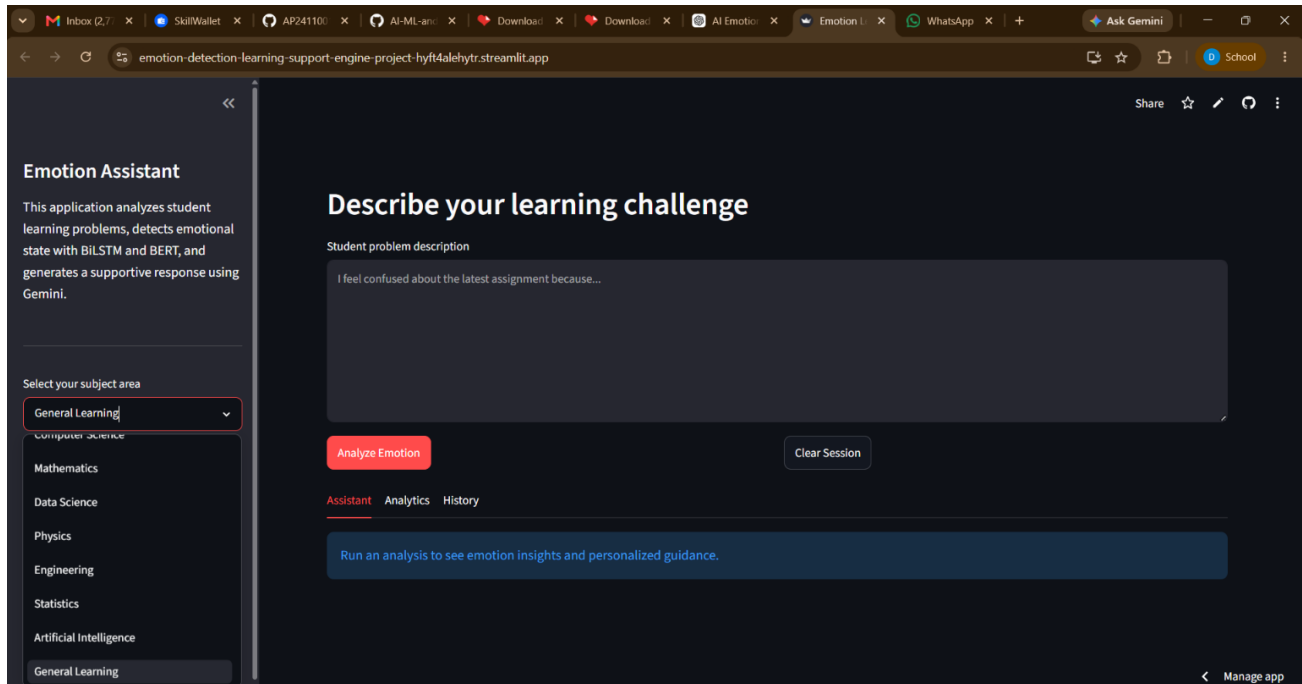
The application was deployed using **Streamlit Community Cloud**, allowing users to access the Emotion Detection & Learning Support Engine through a secure public URL without requiring local installation.

The deployment process included:

- Uploading the complete source code to GitHub.
- Connecting the GitHub repository to Streamlit Community Cloud.
- Configuring the application entry point (app.py).
- Installing project dependencies automatically from requirements.txt.
- Launching the application using Streamlit Cloud.

The deployed application provides public access to all features including:

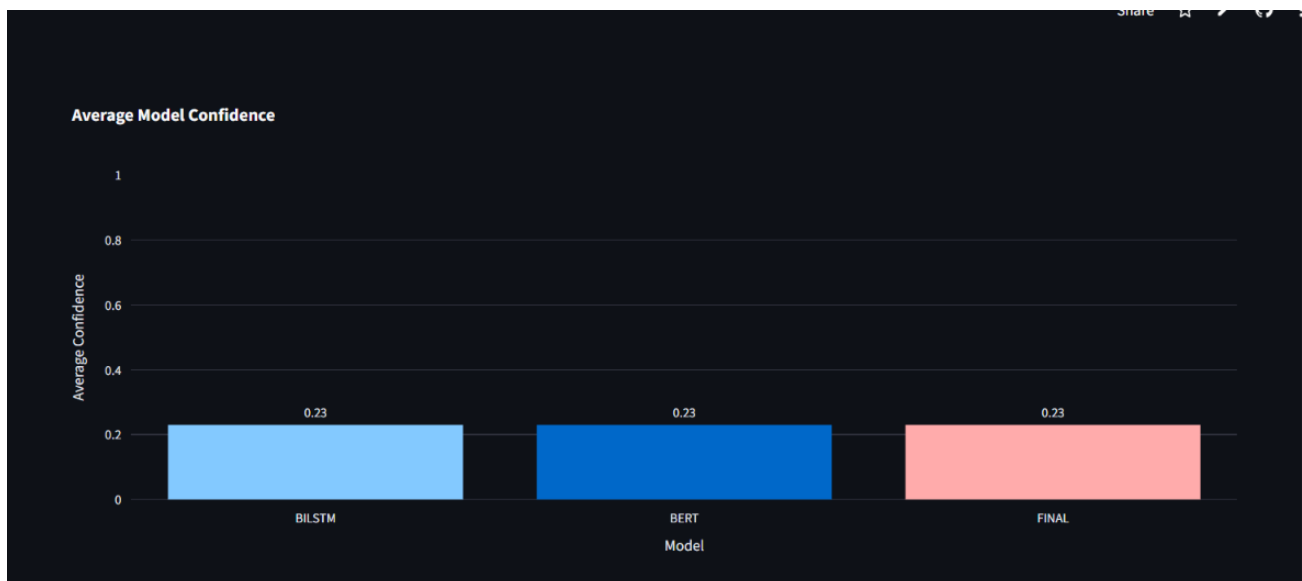
- Emotion Detection
- Personalized Guidance
- Analytics Dashboard
- History Tracking



Home / Assistant Page

Description

The Assistant page serves as the primary interface of the Emotion Detection & Learning Support Engine. Students describe their learning challenges, select their academic subject, and receive emotion analysis together with personalized learning guidance generated using Google Gemini.



Analytics Overview

Total interactions

2

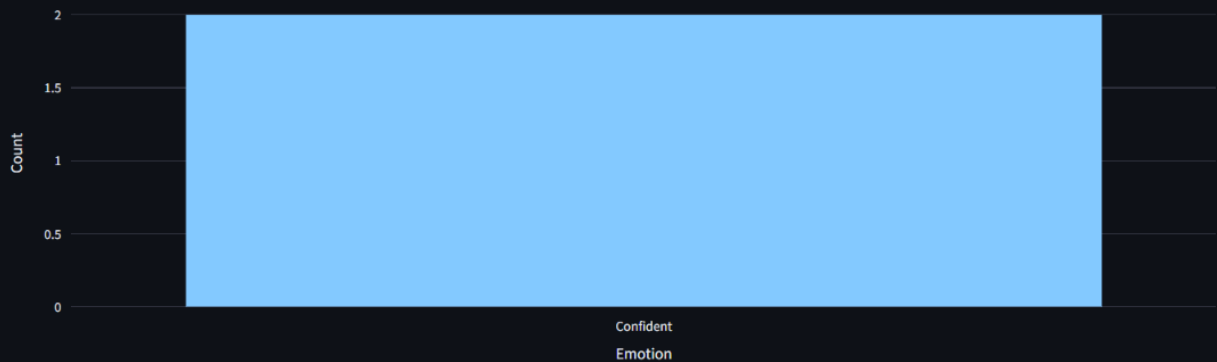
Tracked subjects

2

Average confidence

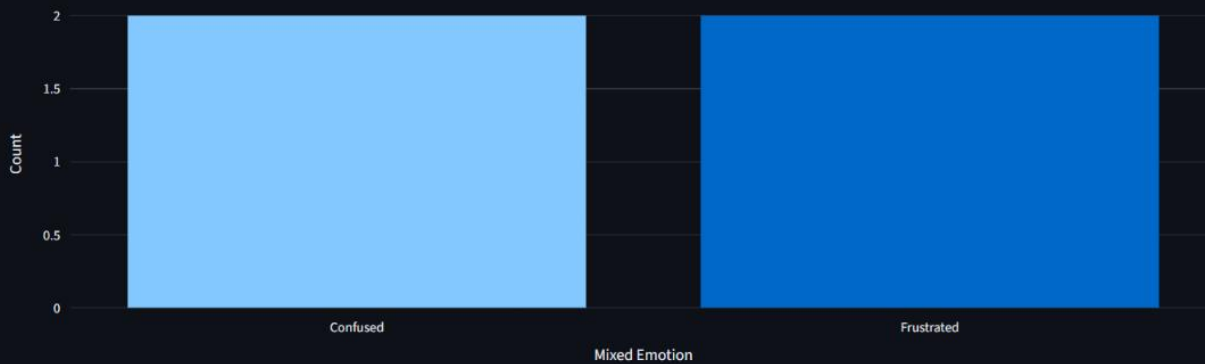
23.00%

Emotion Distribution



< Manage

Mixed Emotion Frequency



Confidence Trend

Analytics Page

Description

The Analytics page visualizes historical emotion prediction data through interactive charts. Users can analyze emotion distribution, confidence trends, model comparison, mixed emotions, and subject-wise statistics.

Session History

	timestamp	student_input	selected_field	bilstm_prediction	bilstm_confidence
0	2026-07-03 14:21:26+00:00	For the past two weeks, I have been preparing for my mathematics exam, especially c	General Learnir	Confident	0.2300
1	2026-07-03 12:06:44+00:00	For the past two weeks, I have been preparing for my mathematics exam, especially c	Mathematics	Confident	0.2300

Download prediction history as CSV

Manage app

History Page

Description

The History page stores all previous interactions in CSV format and allows users to review previous predictions and download prediction history.

Conclusion

The Emotion Detection & Learning Support Engine successfully combines **BiLSTM**, **BERT**, and **Google Gemini** to provide intelligent emotion-aware learning assistance. By analyzing students' textual learning challenges, the system accurately predicts emotional states, identifies mixed emotions, and generates personalized learning guidance in real time. The application also provides comprehensive analytics and interaction history, enabling both students and educators to better understand learning behaviors. Deployment through Streamlit Community Cloud ensures easy accessibility, scalability, and practical usability for modern educational environments.